

Face Recognition Photo Search System

Complete Architecture & Implementation Plan — Local Prototype

Goal	Search 100+ photo stack using 3-4 reference photos of a person
Approach	Face detection + embeddings, precomputed index, fast similarity search
Stack (proto)	Python, InsightFace, FAISS, FastAPI, SQLite/JSON
Stage	Local prototype / proof of concept

1. Objective

Build a system where a stack of 100+ photos can be searched using 3-4 reference photos of a specific person, returning all photos in the stack that contain that person. The system should be fast at query time, reusable across multiple users, and efficient in storage — built first as a local prototype.

2. High-Level Architecture

The system is split into two independent flows:

- **Indexing Flow** — runs once when photos are added to the stack
- **Query Flow** — runs every time a user searches with reference photos

Key principle: face-to-embedding conversion happens only at upload/indexing time, never at query time for the stack. This is what keeps searches fast — at query time only the small reference photo set needs conversion, then it's compared against an already-built index.

3. Tech Stack (Local Prototype)

Component	Choice	Reason
Language	Python	Mature ML ecosystem
Face Detection + Recognition	InsightFace (buffalo_sc / buffalo_s)	Lightweight, accurate, free, runs on CPU
Vector Search	FAISS	Local, fast, free, scales well
Backend API	FastAPI	Matches existing stack
Metadata Storage	SQLite or JSON	Photo path, face ID, person cluster mapping
Parallel Processing	Python multiprocessing / batch loop	Fast bulk indexing
Frontend (testing)	Simple HTML / React page	Upload + query UI

4. Step-by-Step Flow

Phase A — Indexing Pipeline (on stack upload)

Phase B — Query Pipeline (on user search)

Phase C — Result Delivery

5. Incremental Updates

When new photos are added later, the entire stack should not be reprocessed. Only the new photos get embedded and appended to the existing FAISS index, with corresponding new entries added to the metadata store.

6. Threshold Tuning Plan

7. Edge Cases & Problems to Handle

Problem	Solution Plan
No face detected in photo	Skip it, add to a "no-face" list, keep a log
Multiple faces in one photo	Embed each face separately; multiple entries against the same photo_path
Blurry / low quality photo	Check detection confidence score; flag low-confidence detections
Similar-looking people (false positives)	Show top-K results with confidence scores; leave final call to the user
Model change in future	Embeddings from different models are incompatible — switching models means reprocessing
Storage growth	FAISS already handles this better than flat JSON; migrate to Milvus/Pinecone if scale increases
Duplicate photos in stack	Optional: hash-check photos on upload to prevent duplicate entries
Privacy	Keep storage local-only; consider encrypting embeddings if real user data is involved

8. Testing Plan (Localhost Prototype)

9. Future Scaling Considerations

Not needed now, but useful to keep in mind: